

```
!pip install -U langgraph langchain google-generativeai --quiet
```

```
import os, re
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, START
import google.generativeai as genai
```

```
import google.generativeai as genai
genai.configure(api_key="AIzaSyDo3hkd8XPLMFdzQh3HK8xVjT6F60XjLaI")
```

```
def gemini_generate(prompt: str, model_name="gemini-2.0-flash"):
    model = genai.GenerativeModel(model_name)
    resp = model.generate_content(prompt)
    return resp.text.strip() if resp and resp.text else ""
```

```
def add(left, right): return left + right

class State(TypedDict):
    product_query: str
    reviews: Annotated[list[str], add]
    summary: str
    sentiment_score: float
    decision: str
# 6 Node: Product Query
def product_query_node(state: State):
    return {
        "product_query": state["product_query"],
        "reviews": state.get("reviews", []),
    }
```

```
def summary_node(state: State):
    reviews = state.get("reviews", [])
    if not reviews:
        return {"summary": "No reviews available."}
    prompt = f"""
You are a product review summarizer.
Summarize the following reviews in 2-3 lines, listing top pros and cons.

Reviews:
{chr(10).join('- ' + r for r in reviews)}

Summary: """
    try:
        summary = gemini_generate(prompt)
        return {"summary": summary}
    except Exception as e:
        return {"summary": f"Error generating summary: {e}"}
```

```
def reviews_sentiment_node(state: State):
    reviews = state.get("reviews", [])
    if not reviews:
        return {"sentiment_score": 0.0}
    prompt = f"""
You are a sentiment analyzer.
For each review, determine sentiment between -1 (very negative) and +1 (very positive),
then return only ONE average numeric score.
Reviews:
{chr(10).join('- ' + r for r in reviews)}
Average sentiment: """
    try:
        output = gemini_generate(prompt)
        m = re.search(r"(-?\d+(\.\d+)?)", output)
        score = float(m.group(1)) if m else 0.0
        score = max(-1.0, min(1.0, score))
        return {"sentiment_score": score}
    except Exception as e:
        return {"sentiment_score": 0.0, "summary": f"Error generating sentiment: {e}"}
```

```
def should_buy_node(state: State):
    summary = state.get("summary", "")
    score = state.get("sentiment_score", 0.0)
    threshold = 0.5
    decision = "Neutral / Need more info"
    if score >= threshold:
```

```

        decision = " Should Buy"
    elif score <= threshold:
        decision = " Should Not Buy"
    else:
        low = summary.lower()
        if any(k in low for k in ["recommend", "worth", "good", "buy"]):
            decision = " Should Buy"
        elif any(k in low for k in ["avoid", "bad", "poor", "not recommend"]):
            decision = " Should Not Buy"
    return {"decision": decision}

```

```

builder = StateGraph(State)
builder.add_node(product_query_node)
builder.add_node(summary_node)

builder.add_node(reviews_sentiment_node)
builder.add_node(should_buy_node)
builder.add_edge(START, "product_query_node")
builder.add_edge("product_query_node", "summary_node")
builder.add_edge("product_query_node", "reviews_sentiment_node")

builder.add_edge("summary_node", "should_buy_node")
builder.add_edge("reviews_sentiment_node", "should_buy_node")

builder.set_entry_point("product_query_node")
graph = builder.compile()

```

```
review = input()
```

```
iPhone 17
```

```

sample_reviews = [
# 🟢 Positive review
"The battery performance is amazing, and the phone feels super smooth and responsive.",
# Negative review
"Camera quality is disappointing in low light and the device heats up quickly while gaming."
]
input_state = {
"product_query": "iphone 17",
"reviews": sample_reviews
}

result = graph.invoke(input_state)

print("PRODUCT QUERY:", result.get("product_query", "N/A"))
print("\n SUMMARY:\n", result.get("summary", "N/A"))
print("\n SENTIMENT SCORE:", result.get("sentiment_score", "N/A"))
print("\n DECISION:", result.get("decision", "N/A"))

try:
    from IPython.display import Image, display
    display(Image(graph.get_graph().draw_mermaid_png()))
except Exception as e:
    print("Visualization not supported:", e)

```

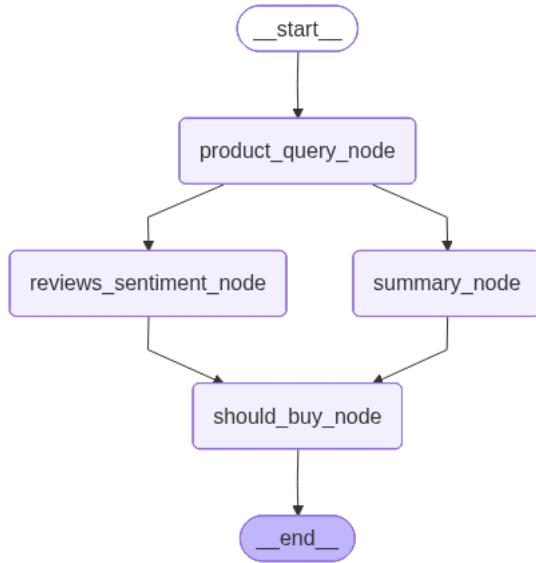
PRODUCT QUERY: iphone 17

SUMMARY:

This phone boasts excellent battery life and a smooth, responsive user experience. However, the camera struggles in low-light

SENTIMENT SCORE: 0.16

DECISION: Should Not Buy



```

!pip install -U langgraph google-generativeai --quiet
import os, re
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, START
import google.generativeai as genai
  
```

```

os.environ["GOOGLE_API_KEY"] = "AIzaSyAcj0or3UwW4YK3EEJ3cx4BwqmtFdJ1vwc" # <-- replace with yours
genai.configure(api_key=os.environ["GOOGLE_API_KEY"])
  
```

```

def gemini_generate(prompt: str, model_name="gemini-2.5-flash"):
    model = genai.GenerativeModel(model_name)
    resp = model.generate_content(prompt)
    return resp.text.strip() if resp and resp.text else ""
  
```

```

def add(left, right): return left + right
class State(TypedDict):
    review: str
    summary: str
    sentiment_score: float
    decision: str
# Node: Review input (entry)
def review_node(state: State):
    return {"review": state["review"]}
# Node: Summary
def summary_node(state: State):
    review = state["review"]
    prompt = f"Summarize this product review in one short sentence:\n\nReview: {review}\n\nSummary:"
    summary = gemini_generate(prompt)
    return {"summary": summary}
# Node: Sentiment
def sentiment_node(state: State):
    review = state["review"]
    prompt = f"""
You are a sentiment analyzer.
Return only one numeric sentiment value between -1 (very negative) and +1 (very positive).
Review: {review}
Sentiment score: """
    output = gemini_generate(prompt)
    m = re.search(r"(-?\d+(\.\d+)?)", output)
    score = float(m.group(1)) if m else 0.0
    score = max(-1.0, min(1.0, score))
    return {"sentiment_score": score}
# Node: Should Buy Decision
def should_buy_node(state: State):
    summary = state["summary"]
  
```

```

score = state["sentiment_score"]
threshold = 0.2
if score >= threshold:
    decision = " Should Buy"
elif score <= -threshold:
    decision = " Should Not Buy"
else:
    decision = " Neutral / Need more info"
return {"decision": decision}

```

```

# Build LangGraph
builder = StateGraph(State)
builder.add_node(review_node)
builder.add_node(summary_node)
builder.add_node(sentiment_node)
builder.add_node(should_buy_node)
builder.add_edge(START, "review_node")
builder.add_edge("review_node", "summary_node")
builder.add_edge("review_node", "sentiment_node")
builder.add_edge("summary_node", "should_buy_node")
builder.add_edge("sentiment_node", "should_buy_node")
builder.set_entry_point("review_node")
graph = builder.compile()
# Take user input
review = input("Enter your product review: ")
# Prepare initial state
input_state = {"review": review}
# Run the graph
result = graph.invoke(input_state)
# Show results
print("\n SUMMARY:", result["summary"])
print(" SENTIMENT SCORE:", result["sentiment_score"])
print(" DECISION:", result["decision"])

```

Enter your product review: Front camera quality is poor

SUMMARY: Front camera quality is poor.
 SENTIMENT SCORE: -0.8
 DECISION: Should Not Buy

graph

